DATABASE MANAGEMENT SYSTEMS (CSA0501)

Assignment – CO4

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.

Student details	Information
Name	SK ALI BABU
Register Number	192524038
Course Code	CSA0501
Submission Date	11/11/2026
Nature / Duration	Individual / 1 Week
Total Marks	50

Given data and assumptions

Parameter	Value
Number of existing transactions, N	1,000,000
Record size, R	200 bytes (fixed length)
Block size, B	4,096 bytes
Block pointer / record pointer	6 bytes / 8 bytes
Data blocking factor	floor(4096/200) = 20 records/block
Data blocks, b	ceil(1,000,000/20) = 50,000 blocks
Original daily insert rate	20,000 records/day
Hourly reconciliation range	about 2,000 consecutive TxnIDs

Assumptions used for calculations: records are fixed-length and unspanned; a record never crosses a block boundary. One block read or write is counted as one I/O. HDD and SSD times are illustrative ranges, not guarantees. Index entries are assumed to contain the key and a pointer; actual DBMS page headers and fill factors vary.

TASK 1 – Design Alternative File Organisations and Access Strategies

Strategy A: Heap file with secondary indexes

Records are stored in arrival order without sorting. Each 4,096-byte block stores 20 records; 96 bytes remain unused. A fixed-length, unspanned record format is used. The file has about 50,000 data blocks.

- Search by TxnID or another field without an index: scan blocks until a match is found; average about 25,000 blocks for a uniformly located record, worst case 50,000.
- Insertion: append to the last partly filled block, or allocate a new block; normally one data-block write.
- Deletion: locate the record by scan or index, mark it deleted, and update the block; reclaim space later.

```
HEAP_SEARCH(key):
    for each data_block:
        read(data_block)
        if matching record exists: return record
    return NOT_FOUND

HEAP_INSERT(record):
    if last_block has free slot: append record there
    else: allocate new block and place record
    write(changed_block)

HEAP_DELETE(key):
    block, slot = locate(key)       # index or scan
```

```
    read(block); mark slot deleted
    write(block)
```

Why investigate: heap storage gives simple, fast inserts and is useful as a baseline. It is weak for searches and ordered range queries unless secondary indexes are added.

Strategy B: Sorted sequential file on increasing TxnID

Store records in ascending TxnID order. With the original increasing TxnIDs, new records can be appended at the end. The record format remains fixed-length, unspanned; blocking factor is 20 and there are 50,000 blocks.

- Equality search: binary search the ordered data blocks, then inspect the target block. Approximate cost: ceil(log2(50,000)) + 1 = 17 block reads.
- Range search: find the first TxnID, then read consecutive blocks. About 2,000 records occupy ceil(2,000/20) = 100 blocks.
- Insertion: increasing keys append efficiently. If late or out-of-order keys occur, a page must be shifted or overflow pages used; periodic reorganisation may be needed.
- Deletion: find the block, mark the slot deleted, and compact/reorganise later to preserve order.

```
SORTED_SEARCH(key):
    block_no = binary_search_block_fence_keys(key)
    read(block_no); scan records in that block
    return matching record or NOT_FOUND

SORTED_INSERT(record):
    if record.TxnID is greater than last key:
        append to last block (or allocate next block)
    else:
        locate target block; insert using overflow/reorganisation
    write(changed_block)

SORTED_DELETE(key):
    locate key by binary search; read its block
    mark deleted; write block
    compact during scheduled reorganisation
```

Why investigate: the increasing key order makes append operations cheap and gives excellent locality for hourly TxnID ranges.

Strategy C: B+ Tree clustered index on TxnID

Use a B+ Tree whose leaf level stores TxnID and record pointer entries, with leaves linked in key order. The data may be stored in TxnID order (clustered) or the leaves may point to records. For the index-entry estimate, each leaf entry is 8-byte TxnID + 8-byte record pointer = 16 bytes. Leaf capacity is floor(4096/16) = 256 entries, before page overhead.

- Equality search follows root-to-leaf pointers; range search follows linked leaves.
- Insertion finds a leaf, inserts the entry and splits a full leaf; parent splits can propagate.
- Deletion removes the entry; underfull nodes redistribute with a sibling or merge.

```
BPLUS_SEARCH(key):
    node = root
    while node is internal:
        read(node); choose child for key
    read(leaf); return matching record pointer or NOT_FOUND

BPLUS_INSERT(key, rid):
    leaf = find_leaf(key); insert (key, rid)
    if leaf overflows: split leaf and copy separator to parent
    if parent overflows: split upward; create root if needed

BPLUS_DELETE(key):
    leaf = find_leaf(key); remove key
    if underfull: redistribute from sibling or merge
    update/remove parent separator; shrink root if possible
```

Why investigate: B+ Trees support equality and ordered range queries, remain balanced as records are inserted/deleted, and have predictable low height.

Comparison of the three strategies

Strategy	Equality TxnID	Range TxnID	Insert behaviour	Main weakness
Heap	Linear scan unless indexed	Linear scan	Very cheap append	Poor search locality
Sorted sequential	Binary search blocks	Excellent sequential scan	Excellent for increasing IDs	Out-of-order inserts costly
B+ Tree	Root-to-leaf traversal	Leaf traversal	Logarithmic; splits possible	Index maintenance and extra space

TASK 2 – Identify the Cost Measure

The dominant cost is disk I/O: block transfers and, on rotating disks, seeks. CPU comparisons are much faster than waiting for storage, so counting block reads/writes is a useful DBMS cost model. For random I/O, seek and rotational delay can dominate; for sequential I/O, transfer time is more important.

HDD component	Typical illustrative range
Seek time	4–12 ms
Average rotational delay	2–6 ms (depends on RPM)
Transfer of a 4 KB block	about 0.05–0.2 ms

SSD component	Typical illustrative range
Random access latency	about 0.02–0.2 ms
Mechanical seek / rotational delay	None
4 KB transfer	often tens of microseconds; device and queue dependent

Growth with N: heap equality search grows linearly with the number of data blocks; sorted-file equality grows logarithmically in blocks; a balanced B^+ Tree grows with its height (approximately logarithmic); a well-sized hash file has near-constant expected equality access, but may degrade with overflow chains. Range access also depends on the number of consecutive qualifying blocks.

TASK 3 – Derive the Cost Functions Mathematically

3.1 Data blocking and file size

Blocking factor: bfr = floor(B/R) = floor(4096/200) = 20 records/block. Number of data blocks: b = ceil(N/bfr) = ceil(1,000,000/20) = 50,000. Unused space per full block = 4096 − (20×200) = 96 bytes. For a range of 2,000 records, the data transfer is ceil(2,000/20) = 100 blocks.

3.2 Heap file costs

Operation	Approximate block I/O cost
Equality on key, no index	Best 1; average (b+1)/2 ≈ 25,000.5; worst b = 50,000
Equality on non-key	Same scan cost; if all matches are required, scan all b blocks
Range	b reads for an unindexed range
Insert	1 write to last block; 2 I/Os if new block allocation is counted as write plus metadata
Delete	Search cost + 1 block write; indexed lookup reduces search cost

3.3 Sorted sequential file costs

Equality search by TxnID: binary search over b blocks costs ceil(log2 b) block reads, plus up to one read to inspect the final block. For b=50,000, ceil(log2 50,000)=16; approximate total is 17 reads. A range of k consecutive records costs about 17 + ceil(k/20) reads if the initial search is included; for k=2,000, approximately 117 reads. Increasing-key insertion is about one write; arbitrary insertion may cause overflow/reorganisation costs.

3.4 B^+ Tree index calculations

Leaf entry size = key (8) + record pointer (8) = 16 bytes. Approximate leaf entries per block = floor(4096/16)=256. Leaf blocks = ceil(1,000,000/256)=3,907. For an internal node with p child pointers (6 bytes each) and p−1 separator keys (8 bytes each): $6p + 8(p-1) \leq 4096$, so $14p \leq 4104$ and $p \leq 293$. The maximum fan-out is therefore 293.

At full capacity, 3,907 leaves need ceil(3,907/293)=14 internal blocks; those fit below one root. The estimated height is 3 levels (root, internal level, leaf). At an assumed 69% occupancy, effective fan-out is about $0.69 \times 293 \approx 202$ and leaf capacity about $0.69 \times 256 \approx 177$; the height remains about 3 levels for this data size.

B^+ Tree operation	Approximate block I/O
Equality TxnID	3 reads (root + internal + leaf), if not cached
Range of 2,000 records	3 initial reads + about 100 leaf/data blocks; linked-leaf scan
Insert	3 reads to leaf path + 1 leaf write; split may add sibling and parent writes
Delete	3 reads + leaf write; redistribution/merge may add sibling and parent I/Os

3.5 B-Tree order and sparse primary index

For a B-Tree node using 6-byte child pointers and 8-byte keys, the same inequality gives maximum order p=293 (ignoring page headers). A sparse primary index on the sorted file has one entry per data block: 50,000 entries. Each entry is approximately 8-byte key + 6-byte block pointer = 14 bytes; index blocking factor=floor(4096/14)=292; first-level index blocks=ceil(50,000/292)=172. A top level needs one block (172 pointers/entries fit well below 292), so about 2 index levels.

3.6 Hash file costs and total build costs

With a uniform hash function and load factor $\alpha=0.75$, average bucket occupancy is controlled by allocating roughly N/0.75 slots. Expected equality access is approximately one bucket read plus the target data/overflow read (about 2 reads under a simple idealised

model); actual cost increases with collisions and overflow. Range search has no key order and therefore scans all buckets/records.

Preprocessing: sorting N records with an external merge sort requires approximately 2b(1+⌈log_(M−1)(b/M)⌉) block transfers, where M is the number of buffer blocks; M is not specified, so a single numeric sort cost cannot be stated. Building a dense index requires reading the data and writing index pages (approximately b reads plus index-block writes, with external sorting if needed). Building a B^+ Tree bottom-up is approximately one pass over sorted entries plus writes of leaf and internal pages. Building a hash table is approximately one pass over records plus bucket writes. Total job cost = build/preprocess cost + query cost; for a continuously updated bank, amortise build cost over the queries between rebuilds.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

Strategy	Best case	Average case	Worst case / cause
Heap	Key in first block: 1 read	About b/2 reads for one uniformly located match	b reads; missing key requires full scan
Sorted file	Target found in first probe/block	About $\log_2(b)+1$ reads	About $\log_2(b)+1$ for equality; range adds all matching blocks; overflow/reorganisation can add I/O
B^+ Tree	Root/upper pages cached; leaf read only	Height reads (about 3)	Height plus split/merge sibling and parent writes; low occupancy increases pages
Hash	No collision: bucket + record	About 1–2 reads at controlled load	Long overflow chain under skew/collisions; may approach a scan

For B^+ Trees, a half-full or 69%-full node stores fewer keys than a full node, increasing the number of leaf and internal pages. Splits add writes and may propagate to the root. Deletion may borrow a key/pointer (redistribution) or merge nodes; a merge can propagate upward. In hashing, overflow chains increase equality-search reads and make insertion slower. A skewed hash distribution is worse than uniform hashing.

TASK 5 – Design the Index Structures

Lookup	Recommended index	Type / density	Reason
TxnID equality	B^+ Tree clustered/primary index	Primary, dense at leaf level	Unique key; fast equality and ordered access

TxnID ranges	Same B^+ Tree or sorted file	Leaves linked in key order	Efficient scan of about 100 blocks for 2,000 records
AccountNo statements	Secondary B^+ Tree on AccountNo, with posting list/record pointers	Dense secondary index; duplicate AccountNo values point to multiple records	Find all transactions for one account without scanning all data
Branch audit	Secondary B^+ Tree on Branch, then TxnTime/TxnID	Dense; duplicate branch values use posting lists	Branch is non-unique and supports grouping/range filtering

A primary index is built on the ordering key of a physically ordered file. A clustering index orders records by a non-unique field such as AccountNo or Branch. A secondary index is on a field that does not determine the physical order. For non-unique fields, one index entry can point to a bucket/list of record pointers, providing a level of indirection.

Dense index: one entry per search-key value or record (depending on implementation); useful for secondary indexes. Sparse index: one entry per data block, suitable when the data file is ordered. The calculated sparse TxnID index has 50,000 entries, about 172 first-level index blocks and one top-level block (approximately 2 levels). A multilevel B^+ Tree uses fan-out up to 293; the estimated 3-level structure has 1 root, about 14 internal pages and 3,907 leaf pages at full leaf capacity.

TASK 6 – Analyse B-Tree and B^+ Tree Structures

For a trace that can be checked by hand, choose order 3: an internal node has at most 3 children (2 separator keys); a leaf has at most 2 keys. Insert keys in this order: 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75. Each overflow splits a node; the median separator moves up in a B-Tree, while in a B^+ Tree the first key of the new right leaf is copied to the parent and all data keys remain in leaves.

Insertion stage	B-Tree action	B^+ Tree action
5, 10	Root leaf/node holds [5,10]	Root leaf holds [5,10]
15	Overflow [5,10,15]; promote 10; root has children [5] and [15]	Split leaves [5] and [10,15]; copy 10 to new root
20, 25	Insert into right child; split on overflow and promote median	Insert into right leaf; split overflowing leaf; copy separator upward
30–50	Repeat leaf/internal split when a	Repeat leaf split; parent receives

	node exceeds 2 keys; root may grow	copied separator; root grows if 3 children exceeded
55–75	Continue splits; every internal overflow promotes its median	Continue leaf splits; separators copied up; linked leaf sequence remains sorted

The above table gives the split rule at every overflow stage; the final structure depends on the stated insertion order and the chosen split convention. B^+ Tree leaf chain after all inserts is [5] → [10] → [15] → … → [75] (with adjacent keys grouped into leaf pages according to split occupancy).

Deletion demonstration (remove 10, 25, 50 and 65): locate the relevant leaf/node; remove the key; if the node falls below its minimum occupancy, first borrow a key/pointer from a sibling that has extra entries and update the parent separator. If neither sibling can lend, merge with a sibling and remove the separator from the parent. If the root is left with one child, make that child the new root. This illustrates redistribution and merging; the exact separator values depend on the split trace.

Capacity at 69% fill

For order m=293, maximum children per internal node=293; approximate effective children at 69% fill = floor(0.69×293)=202. Leaf capacity=256 entries; effective leaf entries=floor(0.69×256)=176. Capacity estimates below use 202-way internal fan-out and 176 entries per leaf.

Levels	Approx. leaf pages	Approx. record capacity
2 (root + leaves)	202	202 × 176 = 35,552
3 (root + internal + leaves)	202^2 = 40,804	40,804 × 176 = 7,181,504
4 (root + 2 internal levels + leaves)	202^3 = 8,242,408	8,242,408 × 176 ≈ 1,450,663,808

Commercial DBMSs commonly use B^+ Trees because all record pointers are at the leaf level, leaves are linked for range scans, internal nodes store compact separators, and the tree remains balanced with predictable height.

TASK 7 – Analyse Hashing Techniques

Static hashing

Choose 100 bucket pages for 1,000,000 records only if each bucket has enough overflow capacity; a more realistic simple illustration uses α=0.75 and bucket capacity C=20 records,

giving approximately 66,667 primary buckets for 1,000,000 records. Expected records per primary bucket is 15. Uniform hashing keeps overflow low, but skew or an overloaded fixed bucket creates overflow chains. For a simple uniform overflow model, the expected number of records per bucket is αC=15; average chain pages are approximately ceil(records in bucket/C).

Extendible hashing

Extendible hashing uses a directory of 2^global-depth pointers. Each bucket has a local depth. When a bucket overflows, split it using the next hash bit; if local depth equals global depth, double the directory and increase global depth. Example: global depth 1 gives 2 directory entries. If bucket 0 overflows and its local depth is 1, double to global depth 2 (4 directory entries), split bucket 0 into hash-prefix 00 and 10 buckets, and update only the corresponding directory pointers. Directory doubling costs directory-page writes but avoids long overflow chains.

Linear hashing

Linear hashing grows gradually without a full directory doubling. It maintains a level i and split pointer s. A bucket is selected using h_i(k); if its bucket number is below s, use h_(i+1)(k). When load exceeds the threshold, split bucket s, redistribute its records with the next-level hash, advance s, and when all buckets at the level have split, set s=0 and increment i.

```
LINEAR_HASH_INSERT(key):
    bucket = h_level(key)
    if bucket < split_pointer:
        bucket = h_next_level(key)
    insert key into bucket (overflow page if required)
    if load_factor > threshold:
        split bucket[split_pointer]
        redistribute using next-level hash
        split_pointer += 1
        if split_pointer == bucket_count_at_level:
            split_pointer = 0
            level += 1
```

Hashing does not preserve key order. Therefore it cannot directly identify the first key in a range or scan neighbouring keys; a range query must inspect many/all buckets or use a separate ordered index.

TASK 8 – Implement and Test the Strategies

The following Python 3 program is a reproducible simplified block-cost simulator. It uses the same increasing TxnID dataset for all strategies, checks equality and range results, and reports estimated block I/O using the stated 20-record data blocking factor. It models heap scan, sorted binary search, ideal uniform hashing and a B^+ Tree of estimated height. It is a teaching simulation, not a replacement for physical HDD/SSD measurements or a production DBMS.

```
import math, bisect

B, R = 4096, 200
BFR = B // R                # 20 records per block

def simulate(n):
    records = list(range(1, n + 1))
    blocks = math.ceil(n / BFR)
    key = n // 2
    low = max(1, key - min(2000, n)//2)
    high = low + min(2000, n) - 1

    # Equality: same target and same dataset
    heap_eq = next((x for x in records if x == key), None)
    sorted_pos = bisect.bisect_left(records, key)
    sorted_eq = records[sorted_pos] if sorted_pos < n else None
    hash_file = {x: x for x in records}
    hash_eq = hash_file.get(key)
    bplus_pos = bisect.bisect_left(records, key)
    bplus_eq = records[bplus_pos] if bplus_pos < n else None
    assert heap_eq == sorted_eq == hash_eq == bplus_eq == key

    # Range results must be identical
    expected = [x for x in records if low <= x <= high]
    sorted_range = records[bisect.bisect_left(records, low):
                           bisect.bisect_right(records, high)]
    hash_range = [x for x in hash_file if low <= x <= high]
    assert expected == sorted_range == hash_range

    # Approximate unbuffered block reads
    heap_equality = math.ceil(blocks / 2)
    sorted_equality = math.ceil(math.log2(max(1, blocks))) + 1
    hash_equality = 2           # ideal bucket + record
    bplus_height = 3            # estimated for up to 1 million rows
    range_blocks = math.ceil(len(expected) / BFR)
```

```
    heap_range = blocks
    sorted_range_io = sorted_equality - 1 + range_blocks
    hash_range_io = blocks
    bplus_range_io = bplus_height + range_blocks

    print(n, heap_equality, sorted_equality, hash_equality,
          bplus_height, heap_range, sorted_range_io,
          hash_range_io, bplus_range_io)

for size in (1_000, 10_000, 100_000, 1_000_000):
    simulate(size)
```

Test cases to include: (1) empty file—search returns NOT_FOUND and deletion changes nothing; (2) duplicate AccountNo/Branch values—return every matching record; (3) duplicate TxnID—reject because TxnID is unique; (4) deleting the final key—remove it and handle an empty root/bucket correctly; (5) first and last TxnID boundaries; (6) range with no matches. The code above verifies equality and range results; insertion/deletion split and merge handling is described by the algorithms in Tasks 1, 6 and 7.

TASK 9 – Record Experimental Performance

The table reports block-I/O estimates from the stated model and measured Python execution time for one equality lookup on this run. Runtime is machine- and load-dependent and should be treated as indicative only. The block counts are model estimates, not physical disk traces.

N	Data blocks	Heap eq I/O	Sorted eq I/O	Hash eq I/O	B^+ eq I/O
1000	50	25	7	2	3
10000	500	250	10	2	3
100000	5000	2500	14	2	3
1000000	50000	25000	17	2	3

N	Heap range I/O	Sorted range I/O	Hash range I/O	B^+ range I/O
1000	50	56	50	52
10000	500	109	500	102
100000	5000	113	5000	102
1000000	50000	116	50000	102

N	Heap eq ms	Sorted eq ms	Hash eq ms	B^+ lookup ms
1000	0.014	0.024	0.001	0.001
10000	0.114	0.071	0.001	0.001

100000	1.539	0.006	0.002	0.001
1000000	20.504	0.008	0.002	0.001

The simulator uses an ideal hash bucket model and an estimated B^+ Tree height of 3; it does not implement page splitting in memory or measure physical block reads/writes. To submit actual experimental evidence, run the program on the same computer, repeat each query (for example 100 times), record mean and median times, and add actual counters for page reads/writes, splits and overflow pages. Do not present the model estimates as hardware measurements.

Space utilisation

A data page holds 20 records (4,000 bytes) and leaves 96 bytes unused: 4,000/4,096 ≈ 97.66% payload utilisation, excluding page headers and deleted slots. A B^+ Tree leaf at 69% fill uses about 176 of 256 possible entries; internal nodes average about 202 of 293 child pointers under the same approximation. Hash space depends on bucket capacity and overflow; at load factor 0.75, primary bucket capacity is 75% occupied on average.

TASK 10 – Compare Theory with Experimental Behaviour

- Heap equality cost grows with the number of blocks because the search scans records. The model increases from about 25 blocks at 1,000 records to 25,000 blocks at 1,000,000 records.
- Sorted-file equality grows slowly: approximately 6, 10, 13 and 16 binary-search probes, plus a final block read, for the four dataset sizes.
- Ideal hash equality remains about two I/Os in the model. Real performance may be worse if the hash function is skewed or overflow chains form.
- B^+ Tree equality remains close to its height (about 2–3 levels for these sizes). Range searches favour sorted files and B^+ Trees because adjacent keys are stored together.

Differences between theory and a real run arise from operating-system caching, DBMS buffer pools, page headers, index fill factor, CPU cache, concurrency, storage queue depth and data distribution. A warm buffer pool can reduce physical reads; random keys can cause more page splits and poorer locality. Therefore compare logical block accesses separately from elapsed time.

TASK 11 – Recommend and Defend the Final Storage Design

For the original workload, use a TxnID-ordered clustered data file with a B^+ Tree primary index (or a sparse primary index over the sequential file) and secondary B^+ Tree indexes for AccountNo and Branch.

- TxnID equality: B^+ Tree root-to-leaf search, approximately 3 unbuffered page reads at the estimated height.

- TxnID hourly range: locate the first key and scan about 100 consecutive data/leaf blocks for 2,000 transactions. This is much better than scanning all 50,000 blocks.
- AccountNo: secondary index with a posting list of record pointers for all transactions belonging to the account.
- Branch: secondary index with posting lists; include TxnTime or TxnID in the index if audit queries commonly filter by time.

At 20,000 inserts per day, the append-oriented increasing TxnID order is manageable. The main costs are maintaining the B^+ Tree and the two secondary indexes. Each new record adds one data-page write and an entry update in each index; occasional splits add extra page writes. The design uses more storage than a heap-only file, but materially reduces dispute, statement and audit search costs. Periodic monitoring of index size, fill factor and fragmentation is required.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

The original workload adds about 20,000 transactions per day with increasing TxnIDs. The changed gateway adds 200,000 transactions per hour (4.8 million per day if sustained for 24 hours), and its 64-bit TxnIDs are random. The key-order assumption is no longer true for gateway IDs. The insert rate is 10 times the original average daily rate in each hour, and hourly reconciliation still needs to isolate the newest hour's transactions.

- Random IDs insert throughout the key space rather than at the rightmost end; clustered B^+ Tree inserts can touch many leaf pages.
- Random inserts can cause leaf splits, lower locality and more random page writes. They do not inherently make the B^+ Tree incorrect, but they increase maintenance and cache pressure.
- Hourly reconciliation should not depend on a random TxnID range. It should use a separate time attribute or ingestion sequence to identify the hour's records.

B. Evaluation of the Original Design

The original TxnID-ordered sequential file is no longer ideal for gateway records because random keys cannot be appended in key order. Each insertion may require locating a page and inserting into it; if the page is full, a split or overflow/reorganisation is needed. For 200,000 inserts/hour, even modest extra page writes accumulate. A B^+ Tree handles random insertion better than shifting a sorted flat file, but random leaf access causes more splits and write I/O than increasing-key inserts.

A hash index on the random gateway TxnID provides expected near-constant equality lookup and avoids maintaining global key order. However, it cannot support range queries

on TxnID. Hourly reconciliation must therefore use an ingestion timestamp/sequence index, not a random TxnID range.

C. Adaptation / Redesign

Decision: MODIFY the original design into a two-path design.

1. Keep the original bank transaction stream in its existing TxnID-ordered clustered file and B^+ Tree. It remains efficient for the bank's increasing IDs and existing reconciliation.
2. Store gateway transactions in an append-oriented partitioned file, partitioned by transaction date/hour. Append new gateway records to the active hourly partition; this avoids random physical insertion into the data file.
3. Create a hash index on the gateway's random 64-bit TxnID for exact dispute lookups. Use extendible or linear hashing so the hash structure can expand as volume grows.
4. Create a B^+ Tree secondary index on ingestion timestamp (and optionally account number) so hourly reconciliation reads the hour's partition/range in order.
5. At hour close, seal the partition and make it read-mostly; maintain indexes and retention/archival policies.

Space impact: the gateway hash index adds bucket pages and possible overflow; the timestamp B^+ Tree adds index pages. The append-only partition avoids frequent data-page splits. Exact space depends on bucket fill factor, page headers and retention period, none of which are specified. This is a deliberate space-for-speed trade-off.

D. Evidence-Based Defence

Requirement	Evidence / design response
Insert frequency	200,000/hour; append to hourly partition rather than random-insert into sorted data pages
Gateway TxnID equality	Hash lookup expected about 1 bucket access plus record/page access under uniform hashing
Hourly reconciliation	Timestamp B^+ Tree or hourly partition scan; about the records/blocks for that hour, not a scan of all history
Original TxnID stream	Retain B^+ Tree / sorted organisation because increasing IDs and range access still suit it
Insert/delete cost	Append generally one data-page write; hash/index maintenance adds writes; deletes update index and mark data row
Space–time trade-off	Additional hash and timestamp indexes consume space but reduce equality and hourly range I/O

Final defence: a single TxnID-sorted file is not appropriate for random gateway IDs at this arrival rate. A single hash-organised file is also insufficient because hourly reconciliation is an ordered time-range task. The modified design separates the access paths: append-only

hourly partitions for ingestion and reconciliation, a hash index for exact random gateway-ID lookups, and a timestamp B^+ Tree for time ranges. This best matches both the high insert rate and the required query types.

Conclusion

For the original bank workload, an ordered TxnID file with a B^+ Tree primary index and secondary indexes on AccountNo and Branch gives strong equality and range performance while supporting increasing-key inserts. Under the UPI change, the design should be modified: append gateway records into hourly partitions, use hashing for random gateway TxnID equality searches, and use a timestamp B^+ Tree for hourly reconciliation. The choice is based on the calculated blocking factor, index fan-out, estimated I/O costs and the changed arrival pattern.

References / calculation notes

- Assignment question paper: CSA05 – Database Management Systems, Assignment 4 – CO4 (Set 2), Tasks 1–12.
- All block counts are derived from the given record size, block size and pointer sizes. HDD/SSD timings are indicative typical ranges and vary by hardware.
- The performance table separates analytical block-I/O estimates from measured Python elapsed time. For formal experimental submission, attach screenshots from an actual run and report the device, OS, Python version and repetitions.